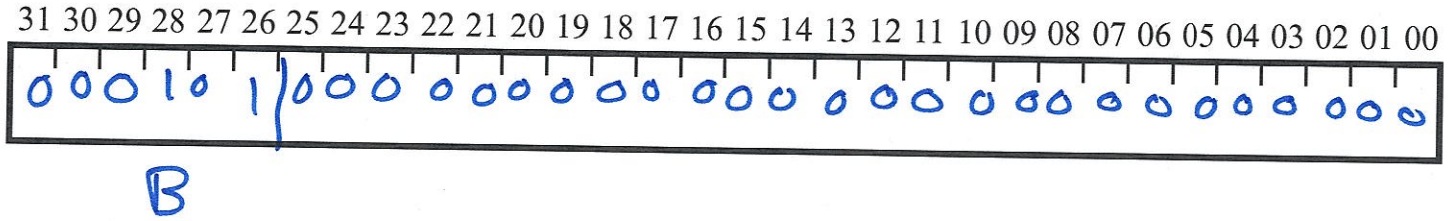
Review Problem 7

- Sometimes it can be useful to have a program loop infinitely. We can do that, regardless of location, by the instruction:
- LOOP: B LOOP
- Convert this instruction to machine code



Assembly & Machine Language

Assembly

Simple instructions
Human-"friendly"
(Almost) 1-to-1 relationship to machine language

Machine Language

Numeric representation of instructions
Fixed format(s), simple to decode
Based on HW

Computer Arithmetic

Readings: 3.1-3.3, A.5

Review binary numbers, 2's complement

fast

Develop Arithmetic Logic Units (ALUs) to perform CPU functions.

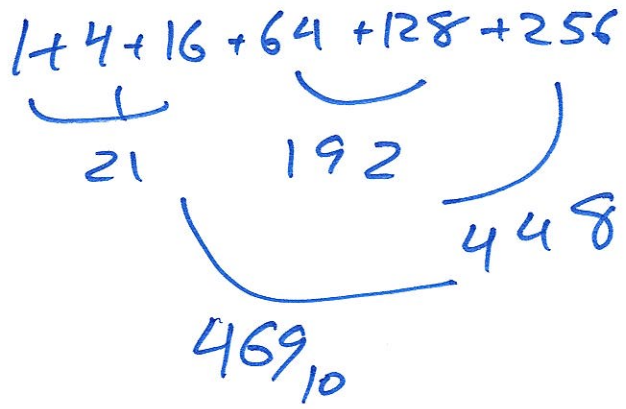
Introduce shifters, multipliers, etc.

Binary Numbers

Decimal: $469 = 4*10^2 + 6*10^1 + 9*10^0$

Binary: $01101 = 1*2^3 + 1*2^2 + 0*2^1 + 1*2^0 = (13)_{10}$

Example: $0111010101 = (?)_{10}$



2's Complement Numbers

Positive numbers & zero have leading 0, negative have leading 1

Negation: Flip all bits and add 1

Ex: $-(01101)_2 =$ $10010 + 1 = 10011_2$

To interpret numbers, convert to positive version, then convert:

$11010 =$

$= - - (11010)$

$= -(-11010)$

$= -(00101+1)$

$= -(00110)$

$= -6_{10}$

$01100 = +12_{10}$

Sign Extension

Conversion of n-bit to (n+m)-bit 2's complement: replicate the sign bit

$b_3b_2b_1b_0 = b_3b_3b_3b_2b_1b_0 = b_3b_3b_3b_3b_3b_3b_3b_3b_3b_3b_3b_2b_1b_0$

Ex - Convert to 8-bit: $01101 = (13)_{10}$ $\quad 11101 = (-3)_{10}$

000 01101

111 11101 ?-3?

$= -(-11111101)$

$= -(00000010+1)$

$= -(00000011)$

$= -3_{10}$

Arithmetic Operations

Decimal:

$$\begin{array}{r} 1\ 1\ 1\ 0 \\ 5\ 7\ 8\ 9\ 2 \\ +\ 7\ 8\ 9\ 5\ 6 \\ \hline 3\ 6\ 8\ 4\ 8 \end{array}$$

Odometer math

Binary:

$$\begin{array}{r} 0\ 0\ 0\ 1\ 1\ 1 \\ 1\ 0\ 1\ 0\ 1\ 1\ 1 \\ +\ 0\ 1\ 0\ 0\ 1\ 0\ 1 \\ \hline 1\ 1\ 1\ 1\ 1\ 0\ 0 \end{array}$$

Binary:

$$\begin{array}{r} 1\ 0\ 1\ 0\ 0\ 1\ 1\ 0 \\ -\ 0\ 0\ 0\ 1\ 0\ 1\ 1\ 1 \end{array}$$

flip

$$\begin{array}{r} 1\ 1\ 0\ 0\ 0\ 0\ 0\ 1 \\ 1\ 0\ 1\ 0\ 0\ 1\ 1\ 0 \\ +\ 1\ 1\ 1\ 0\ 1\ 0\ 0\ 0 \\ \hline 1\ 0\ 0\ 0\ 1\ 1\ 1\ 0 \end{array}$$

add 1

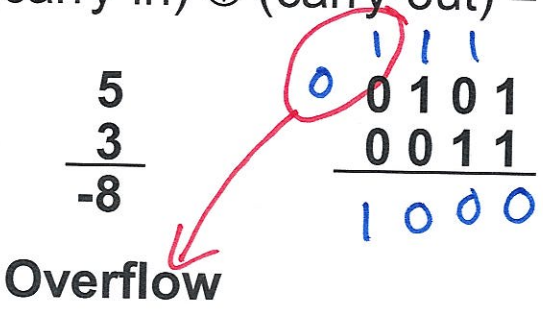
Overflows

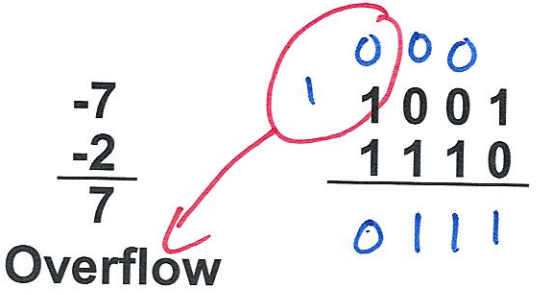
Operations can create a number too large for the number of bits

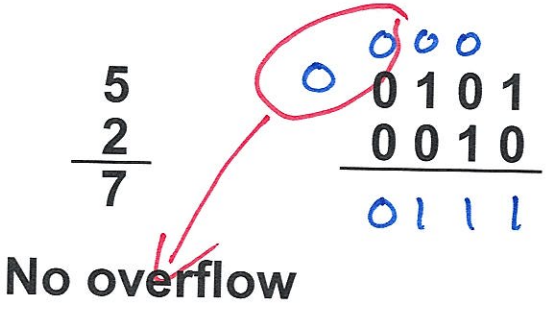
n-bit 2's complement can hold $-2^{(n-1)} \ldots 2^{(n-1)}-1$

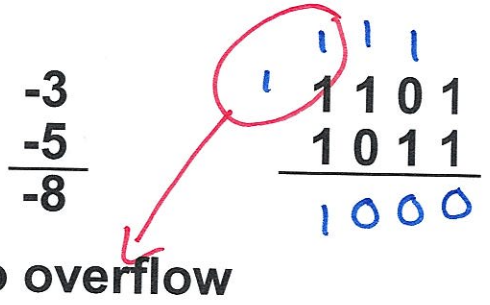
Can detect overflow in addition when highest bit has carry-in ≠ carry-out

(carry-in) ⊕ (carry-out) = 1









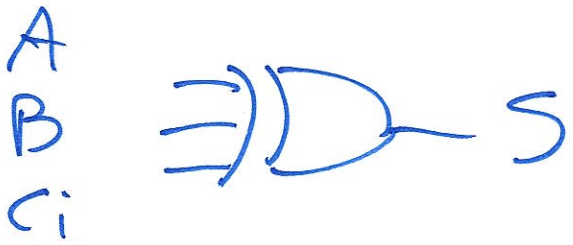
Mixed Sign add

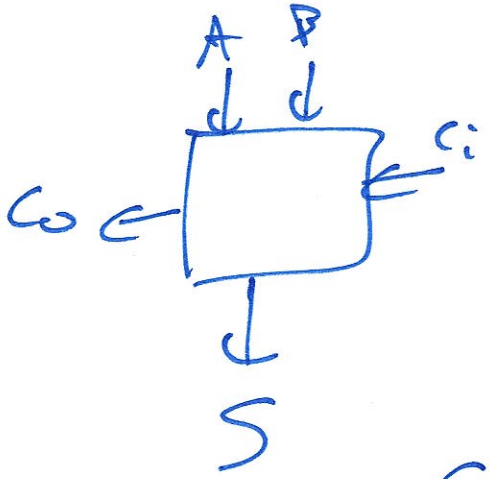
0 xxxx
1 xxxx

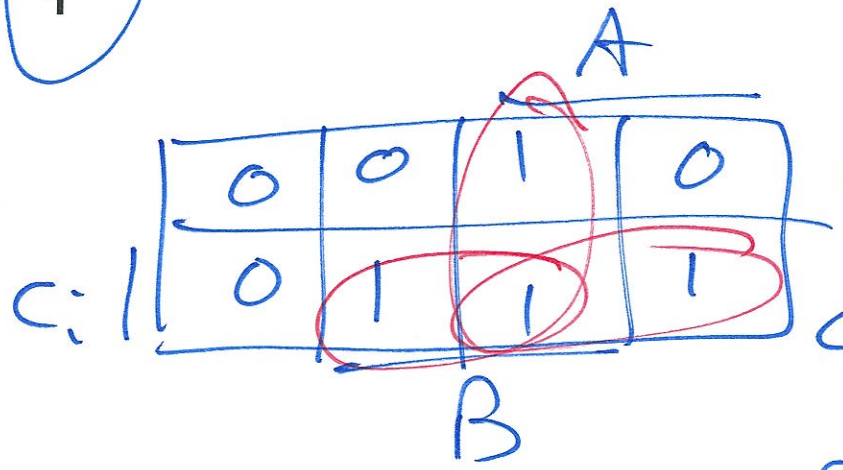
Full Adder

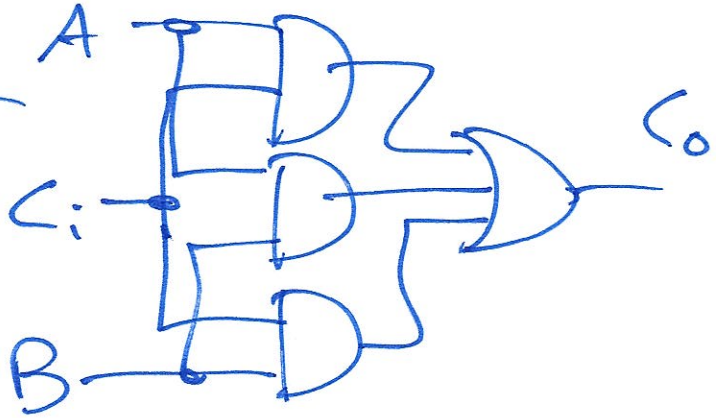
3-input XOR

A	B	CI	CO	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1





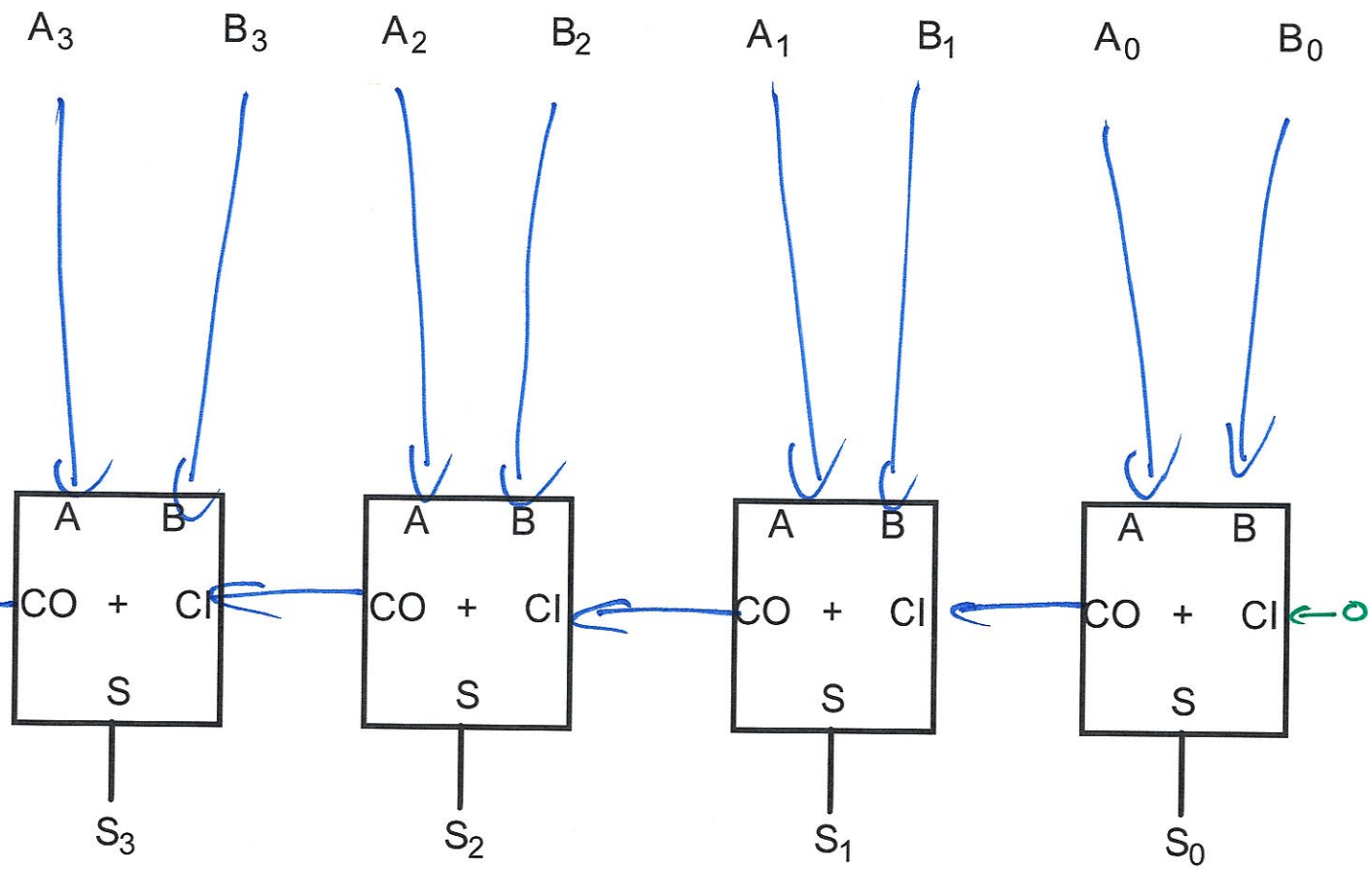




$CO = AB + AC_i + BC_i$

Multi-Bit Addition

$$\begin{array}{r} A_3\ A_2\ A_1\ A_0 \\ +\ B_3\ B_2\ B_1\ B_0 \\ \hline S_3\ S_2\ S_1\ S_0 \end{array}$$



Adder/Subtractor

Subtract

0: A + B = = A + B + 0

1: A − B = A + (−B) = A + $\bar{B}$ + 1

